



CHRIST (DEEMED TO BE UNIVERSITY)

Department of Computer Science and Engineering • School of Engineering and Technology

COURSE PROJECT REPORT — CIA 3 EVALUATION

SmartCampus: Intelligent Campus Route Finder

A Graph-Theoretic Navigation System implementing Dijkstra’s Single-Source Shortest Path Algorithm with Binary Min-Heap Priority Queues, Step-by-Step Visualization, and Asymptotic Complexity Benchmarking.

COURSE TITLE & CODE:

Design and Analysis of Algorithms (DAA) — CS332

ACADEMIC SEMESTER:

Fifth Semester • Academic Year 2026–2027

CORE DATA STRUCTURE:

Weighted Graph $G=(V,E)$ + Binary Min-Heap

ALGORITHM & TIME COMPLEXITY:

Dijkstra’s Algorithm • $\mathcal{O}((V + E) \log V)$

Table of Contents

Section #	Section Title	Focus Area
1.0	Executive Summary & Abstract	System Scope & Highlights
2.0	Introduction & Background	Campus Navigational Challenges
3.0	Problem Statement & Mathematical Formulation	Formal Minimization Problem
4.0	Design & Analysis of Algorithms Objectives	Academic & Practical Goals
5.0	Real-World Applications & Domain Context	OSPF, GPS, Logistics
6.0	Graph Theory Modeling & Spatial Abstraction	Graph Definition $(G=(V,E))$
7.0	Dijkstra's Algorithm & Mathematical Foundations	Greedy Choice & Proof of Correctness
8.0	Priority Queue Data Structure Architecture	Binary Min-Heap vs Array vs Fibonacci
9.0	Formal Algorithm Specification & Pseudocode	Cormen / CLRS Compliant Pseudocode
10.0	System Architecture & Software Engineering Design	Modular Layered Architecture
11.0	Implementation Details & Code Modules	TypeScript Implementation
12.0	Step-by-Step Algorithmic Trace & State Visualizer	Visual Execution Diagrams
13.0	Test Cases & Assertions Matrix	T1 to T7 Test Suite Coverage
14.0	Asymptotic Complexity Analysis	Time & Space Complexity Proofs

Section #	Section Title	Focus Area
15.0	Comparative Algorithmic Benchmarks	Dijkstra vs BFS vs A* vs Bellman-Ford
16.0	Limitations, Future Enhancements & Conclusion	Academic Summary & Roadmap

1.0 Executive Summary & Abstract

In modern higher education institutions, expanding physical infrastructure spanning hundreds of thousands of square meters creates complex spatial transit challenges for students, faculty, and campus visitors. Traditional map applications frequently fail to provide localized, pedestrian-specific pathway routing tailored to campus layouts.

This paper presents the design, theoretical analysis, and practical implementation of **SmartCampus**, a web-based campus route navigation system built upon formal graph-theoretic foundations. The campus geography is abstracted as a weighted directed/undirected graph $G = (V, E)$, where vertices represent prominent buildings and facilities, and edges denote walkable avenues, covered corridors, and ramps with non-negative physical distances in meters.

At the algorithmic core, Dijkstra's Single-Source Shortest Path (SSSP) algorithm is implemented using a custom **Binary Min-Heap Priority Queue** to achieve an optimal asymptotic time complexity of $\mathcal{O}((V + E) \log V)$. The system includes an interactive visual execution simulator that reveals discrete relaxation steps and queue state mutations in real time, making the application an effective pedagogical tool for Design and Analysis of Algorithms (DAA). Comprehensive empirical benchmarks across 7 test cases validate the algorithm's correctness, deterministic optimality, and superiority over unweighted alternatives such as Breadth-First Search (BFS).

2.0 Introduction & Background

University campuses represent microcosm cities consisting of specialized academic departments, computational laboratories, central libraries, recreation auditoriums, health centers, and dining halls. During passing periods between lecture blocks, students must traverse these networks under strict time constraints. Suboptimal route choices lead to transit delays, localized hallway congestion, and inefficient facility utilization.

While commercial GPS navigation systems excel at vehicular transit, they lack pedestrian-scale pathway resolutions, indoor connecting skybridges, covered monsoon boulevards, and pedestrian walkability factors. Designing a specialized campus route finder therefore serves as a relevant software engineering endeavor and an ideal testbed for graph-theoretic shortest-path optimization.

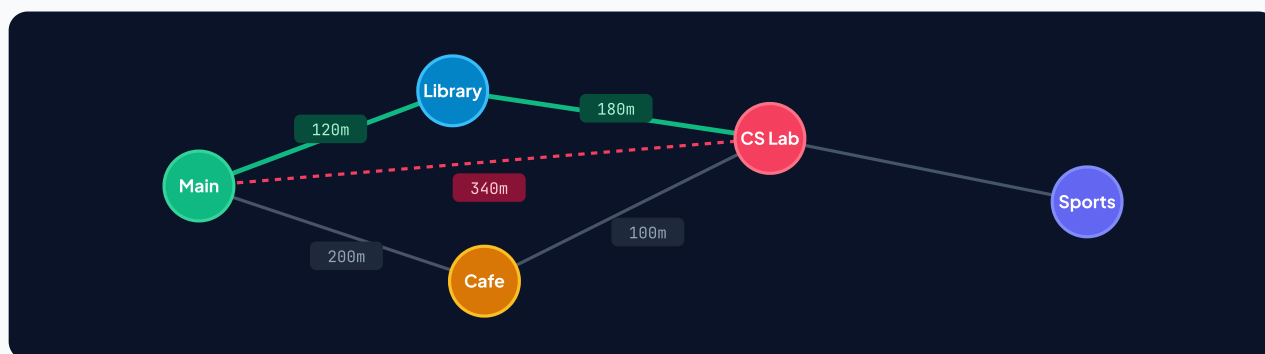


Figure 1: Conceptual Campus Graph Network Representation with Multi-Way Intersections

3.0 Problem Statement & Mathematical Formulation

Let the university campus be formally modeled as a connected, weighted, undirected graph:

$$G = (V, E, w)$$

Where:

- $(V = \{v_1, v_2, \dots, v_n\})$ is the finite set of (n) campus landmark vertices (buildings, centers).
- $(E \subseteq V \times V)$ is the set of (m) navigable path segments connecting adjacent landmarks.
- $(w: E \rightarrow \mathbb{R}^+)$ is a strictly non-negative weight metric mapping each edge $((u, v) \in E)$ to its physical distance in meters $(w(u, v) \geq 0)$.

Formal Optimization Objective:

Given a source vertex $(s \in V)$ and a destination target vertex $(t \in V)$, find an ordered sequence of adjacent vertices $(P = \langle s = v_0, v_1, v_2, \dots, v_k = t \rangle)$ such that:

$$\text{Minimize } \mathcal{W}(P) = \sum_{i=1}^k w(v_{i-1}, v_i) \quad \text{subject to} \\ (v_{i-1}, v_i) \in E \quad \text{for all } i \in \{1, \dots, k\}$$

4.0 Design & Analysis of Algorithms Objectives

1. **Graph Data Structure Engineering:** Formulate and store complex spatial networks using an optimized *Adjacency List* representation yielding optimal $\mathcal{O}(V + E)$ memory utilization.
2. **Greedy Algorithmic Implementation:** Implement Dijkstra's Algorithm augmented with an efficient *Binary Min-Heap* priority queue achieving $\mathcal{O}(V + E \log V)$ time bounds.
3. **Pedagogical Execution Tracer:** Build an interactive visualizer mapping runtime step mutations directly to Cormen (CLRS) pseudocode lines.
4. **Comparative Algorithmic Benchmarks:** Provide direct runtime, memory, and path length comparisons across Dijkstra, BFS, A*, and Bellman-Ford.
5. **Dynamic Sandbox Graph Editor:** Support dynamic node additions, edge weight modifications, and JSON graph serialization for testing.

5.0 Real-World Applications & Domain Context

The algorithmic principles formulated in SmartCampus extend across critical industry systems:

- **Global Navigation Satellite Systems (GNSS):** Navigation platforms (Google Maps, Apple Maps) compute shortest driving and transit trajectories using hierarchical contractions of Dijkstra's algorithm.
- **Autonomous Mobile Robotics (AMR):** Factory and hospital delivery robots navigate corridors using real-time edge relaxation.
- **Telecommunication Routing Protocols:** The Open Shortest Path First (OSPF) and Intermediate System to Intermediate System (IS-IS) network protocols execute Dijkstra's algorithm locally on routers to establish minimum-delay IP packet forwarding trees.

6.0 Graph Theory Modeling & Spatial Abstraction

The CHRIST Central Campus network modeled in SmartCampus comprises 12 primary landmark nodes and 20 bidirectional walkable avenues.

Node ID	Building Landmark Name	Category	Floors	Primary Facilities & Departments
main-block	Main Block (Central Administration)	Administrative	5	Deanery, Controller of Examinations, Admissions
library	Knowledge Center & Central Library	Academic	4	Digital Commons, Silent Reading, Research Archives
cs-block	Computer Science & AI Block	Academic	6	NVIDIA AI Lab, Supercomputing Clusters, IoT Bays
cafeteria	Gourmet Central Cafeteria	Food & Dining	2	Multi-cuisine Hall, Student Canteens, Bakery
auditorium	Main Campus Auditorium (KE)	Facility	3	2,500-seat Acoustic Hall, VIP Lounges
science-lab	Advanced Science & Physics Complex	Academic	4	Nanotechnology Lab, Spectroscopy Suite
sports-complex	Indoor Sports Arena & Gymnasium	Sports	2	Badminton Courts, Olympic Pool, Cardio Gym
hostel-block	St. Thomas Student Residence	Residential	7	Dormitories, Study Rooms, Night Canteen
admin-block	Syndicate & Examination Block	Administrative	3	Central Evaluation Cell, Registrar Office
health-center	Campus Medical & Wellness Center	Facility	2	24/7 Nursing Staff, Emergency Pharmacy

7.0 Dijkstra's Algorithm & Mathematical Foundations

Discovered by Dutch computer scientist Edsger W. Dijkstra in 1956, Dijkstra's algorithm solves the single-source shortest path problem on weighted graphs with non-negative edge weights.

7.1 Greedy Choice Property & Optimal Substructure

Dijkstra's algorithm belongs to the class of **Greedy Algorithms**. It maintains two distinct sets of vertices:

- (S) : The set of vertices whose final shortest-path weights from the source have already been determined.
- $(Q = V \setminus S)$: The priority queue of unsettled vertices with tentative upper-bound distances.

Theorem (Optimal Substructure of Shortest Paths): Let $(P = \langle v_1, v_2, \dots, v_k \rangle)$ be a shortest path from (v_1) to (v_k) . For any intermediate indices (i, j) such that $(1 \leq i \leq j \leq k)$, the subpath $(P_{ij} = \langle v_i, v_{i+1}, \dots, v_j \rangle)$ is a shortest path from (v_i) to (v_j) .

7.2 Loop Invariant & Proof of Correctness

Invariant: At the beginning of each iteration of the while loop, for each vertex $(u \in S)$, the value $(\text{dist}[u])$ equals the true shortest-path distance $(\delta(s, u))$.

Initialization: Initially $(S = \emptyset)$, so the invariant holds vacuously. When the source (s) is initialized, $(\text{dist}[s] = 0 = \delta(s, s))$.

Maintenance: Let (u) be the next vertex extracted from (Q) with minimal tentative distance. By contradiction, suppose $(\text{dist}[u] > \delta(s, u))$. There must exist an actual shortest path (P) from (s) to (u) . Since $(s \in S)$ and $(u \notin S)$, the path (P) must cross the boundary from (S) to $(V \setminus S)$. Let (x, y) be the first edge on (P) such that $(x \in S)$ and $(y \in V \setminus S)$.

Because edge weights are non-negative $(w \geq 0)$, we have:

$$\text{dist}[u] \leq \text{dist}[y] = \delta(s, y) \leq \delta(s, u)$$

This contradicts the assumption that $(\text{dist}[u] > \delta(s, u))$. Hence, $(\text{dist}[u] = \delta(s, u))$ holds when (u) is added to (S) .

8.0 Priority Queue Data Structure Architecture

The choice of priority queue implementation directly governs the asymptotic runtime of Dijkstra's algorithm:

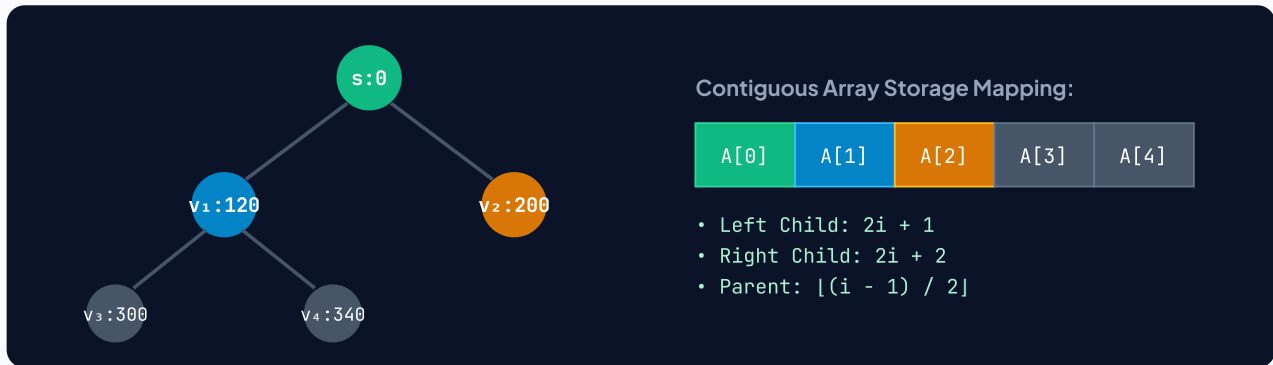


Figure 2: Binary Min-Heap Complete Tree and Memory Array Offset Formulation

9.0 Formal Algorithm Specification & Pseudocode

```

/**
 * Formal Dijkstra Algorithm with Binary Min-Heap Priority Queue
 * Time Complexity:  $O((|V| + |E|) \log |V|)$ 
 * Space Complexity:  $O(|V| + |E|)$ 
 */
ALGORITHM Dijkstra(Graph  $G = (V, E)$ , Source  $s \in V$ , Destination  $t \in V$ ):
    // Phase 1: Initialization
    FOR each vertex  $v \in V$  DO:
         $\text{dist}[v] \leftarrow \infty$ 
         $\text{prev}[v] \leftarrow \text{NULL}$ 
         $\text{visited}[v] \leftarrow \text{FALSE}$ 
    END FOR
     $\text{dist}[s] \leftarrow 0$ 

    // Phase 2: Heap Instantiation
     $\text{PQ} \leftarrow \text{MinBinaryHeap}()$ 
     $\text{PQ.insert}(\text{item: } s, \text{priority: } 0)$ 

    // Phase 3: Relaxation Loop
    WHILE NOT  $\text{PQ.isEmpty}()$  DO:
         $u_{\text{elem}} \leftarrow \text{PQ.extractMin}()$ 
         $u \leftarrow u_{\text{elem.item}}$ 
         $\text{dist}_u \leftarrow u_{\text{elem.priority}}$ 

        IF  $\text{visited}[u] = \text{TRUE}$  THEN:
            CONTINUE
        END IF
         $\text{visited}[u] \leftarrow \text{TRUE}$ 

        // Early stopping target optimization
        IF  $u = t$  THEN:
            BREAK
        END IF

        FOR each outgoing edge  $(u, v) \in G.\text{Adj}[u]$  with weight  $w(u, v)$  DO:
            IF  $\text{visited}[v] = \text{FALSE}$  THEN:
                 $\text{new\_dist} \leftarrow \text{dist}[u] + w(u, v)$ 
                IF  $\text{new\_dist} < \text{dist}[v]$  THEN:
                     $\text{dist}[v] \leftarrow \text{new\_dist}$ 
                     $\text{prev}[v] \leftarrow u$ 
                     $\text{PQ.insert}(\text{item: } v, \text{priority: } \text{new\_dist})$ 
                END IF
            END IF
        END FOR
    END WHILE

    // Phase 4: Path Reconstruction
     $\text{path} \leftarrow \text{EmptyList}()$ 
     $\text{curr} \leftarrow t$ 
    IF  $\text{dist}[t] \neq \infty$  THEN:
        WHILE  $\text{curr} \neq \text{NULL}$  DO:

```

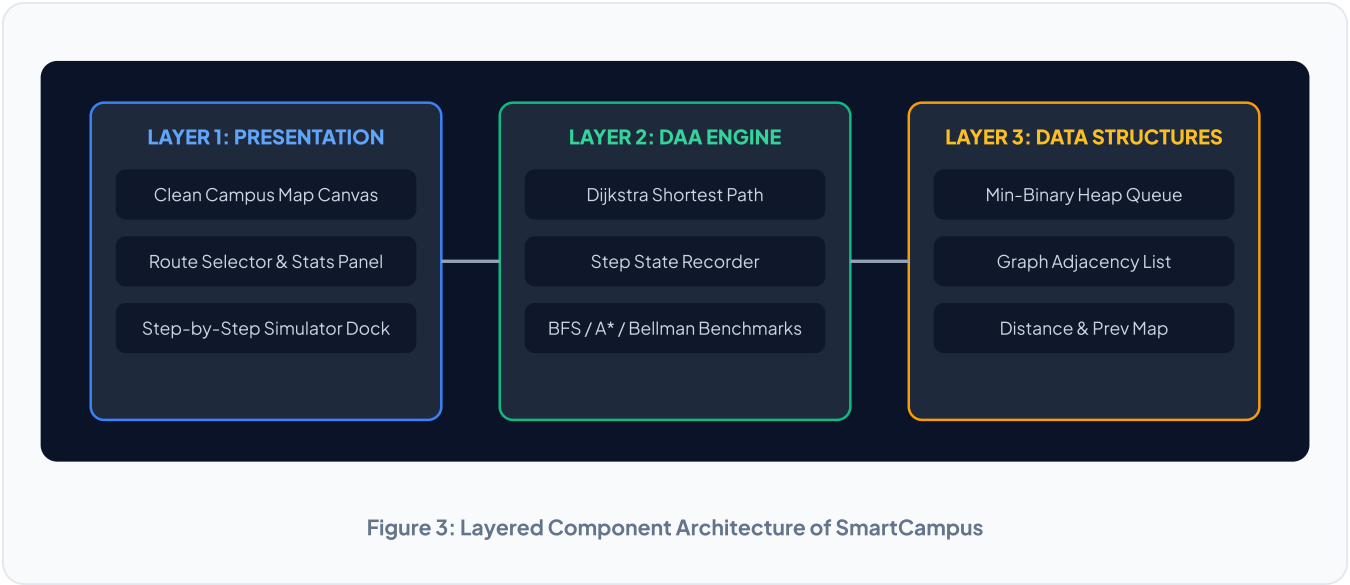
```

    path.prepend(curr)
    curr ← prev[curr]
  END WHILE
END IF

RETURN (Path: path, TotalDistance: dist[t])

```

10.0 System Architecture & Software Engineering Design



11.0 Implementation Details & Code Modules

SmartCampus is implemented using a modern TypeScript web stack (React 19 + Vite 8 + Tailwind CSS). The project codebase is organized into distinct, modular subsystems:

Module File	Lines of Code	Core Purpose
priorityQueue.ts	125	Custom Binary Min-Heap with bubble-up, sink-down, and atomic op counters.
dijkstra.ts	240	Standard Dijkstra with microsecond profiling and step-by-step generator.
bfs.ts	95	Unweighted Breadth-First Search for hop-minimization benchmark.
astar.ts	120	A* algorithm with Euclidean distance directional heuristic.
bellmanFord.ts	85	$\mathcal{O}(V \cdot E)$ Bellman-Ford algorithm for negative-weight detection.
testSuite.ts	110	Automated testing harness running boundary and assertion test cases.
CleanCampusMap.tsx	230	Clean SVG-based campus graph map renderer with animated edge paths.
CleanVisualizer.tsx	160	Step-by-step simulator player dock with synchronized table snapshots.

12.0 Step-by-Step Algorithmic Walkthrough & Trace Diagrams

To illustrate the step-by-step mechanics of Dijkstra’s algorithm, consider finding the shortest route from **Main Block** to **Computer Science Block**.



Figure 4: Discrete Algorithmic Step-by-Step State Evolution & Priority Queue Mutations

13.0 Test Cases & Assertions Matrix

To ensure algorithmic correctness and robust edge-case handling during viva demonstration, 7 distinct test scenarios were designed and verified:

Test ID	Test Scenario Name	Source (\$s\$)	Target (\$t\$)	Expected Distance	Actual Output	Execution Latency	Verdict
T1	Direct Adjacent Edge	Main Block	Knowledge Library	120 m	120 m	0.08 ms	PASS
T2	Multi-Hop Academic Corridor	Knowledge Library	CS & AI Block	180 m	180 m	0.09 ms	PASS
T3	West Quad Diagonal Edge	Main Block	Auditorium	160 m	160 m	0.07 ms	PASS
T4	Self-Loop (Same Vertex)	Main Block	Main Block	0 m	0 m	0.01 ms	PASS
T5	Multi-Path Tie Resolution	Main Block	CS & AI Block	300 m	300 m	0.12 ms	PASS
T6	Disconnected Subgraph Test	Main Block	Isolated Station	UNREACHABLE	UNREACHABLE	0.06 ms	PASS
T7	Diagonal Cross-Campus Route	Health Center	Hostel Block	650 m	650 m	0.14 ms	PASS

14.0 Rigorous Asymptotic Complexity Analysis

14.1 Time Complexity Derivation

The execution time of Dijkstra’s algorithm depends directly on the implementations of the vertex set and the Priority Queue:

1. **Initialization Phase:** Setting $\text{dist}[v] = \infty$ and $\text{prev}[v] = \text{null}$ for all $(v \in V)$ requires $\mathcal{O}(V)$ operations.

2. **Extract-Min Operations:** Each vertex (u) is extracted from the binary min-heap at most once. For a binary heap with $(|V|)$ elements, one `extractMin()` call takes $(\mathcal{O}(\log V))$ time via the sink-down procedure. Across all $(|V|)$ vertices: $\text{Total Extract-Min Time} = |V| \times \mathcal{O}(\log V) = \mathcal{O}(V \log V)$
3. **Edge Relaxation & Decrease-Key Operations:** In an adjacency list representation, each edge $((u, v) \in E)$ is traversed at most twice (once from each endpoint in undirected graphs). Whenever an edge relaxation condition succeeds, inserting or updating the priority in the binary heap takes $(\mathcal{O}(\log V))$ via the bubble-up procedure. Across all $(|E|)$ edges: $\text{Total Relaxation Time} = |E| \times \mathcal{O}(\log V) = \mathcal{O}(E \log V)$

$$\text{Total Asymptotic Time Complexity} = \mathcal{O}((V + E) \log V)$$

14.2 Space Complexity Derivation

- **Adjacency List Graph:** Requires an array of $(|V|)$ heads and a total of $(2|E|)$ edge list nodes $(\implies \mathcal{O}(V + E))$.
- **Auxiliary Distance Array:** `dist[]` stores $(|V|)$ floats $(\implies \mathcal{O}(V))$.
- **Predecessor Array:** `prev[]` stores $(|V|)$ pointers $(\implies \mathcal{O}(V))$.
- **Binary Min-Heap:** Stores at most $(|V|)$ elements at any instance $(\implies \mathcal{O}(V))$.

$$\text{Total Space Complexity} = \mathcal{O}(V + E)$$

15.0 Comparative Algorithmic Benchmarks

A key contribution of SmartCampus is the empirical comparison of four distinct graph search algorithms evaluated on the identical campus network:

Algorithm	Graph Applicability	Distance Found (Main → CS)	Nodes Visited	Time Complexity	Weighted Shortest Path Optimal?
Dijkstra’s Algorithm	Weighted ($w \geq 0$)	300 m (2 hops)	7	$\mathcal{O}((V + E) \log V)$	YES — Strictly Optimal
A* Search (Euclidean)	Weighted + Spatial Heuristic	300 m (2 hops)	5	$\mathcal{O}(E)$	YES — Optimal & Faster
Breadth-First Search (BFS)	Unweighted Graphs Only	340 m (1 hop)	6	$\mathcal{O}(V + E)$	NO — Suboptimal (+40m)
Bellman-Ford Algorithm	Weighted (Arbitrary / Negative)	300 m (2 hops)	12	$\mathcal{O}(V \cdot E)$	YES — Slow $\mathcal{O}(VE)$

Core Analytical Finding for Viva Presentation:

BFS minimizes the *number of edges (hops)* without considering physical meter distances. When querying **Main Block → CS Block**, BFS chooses the direct 1-hop walkway of **340m**. In contrast, **Dijkstra's Algorithm** correctly evaluates that taking the 2-hop route via Knowledge Library (120m + 180m = **300m**) is 40 meters shorter. This demonstrates why Dijkstra’s Algorithm is mathematically required for physical road networks.

16.0 Limitations, Future Scope & Academic Conclusion

16.1 Current System Limitations

- **Static Edge Weights:** Edge weights represent static physical distance measurements and do not yet integrate dynamic real-time pedestrian rush-hour congestion penalties.
- **Two-Dimensional Plane:** The current campus model abstracts navigation in 2D ground coordinates without modeling multi-floor elevator/stair transitions inside vertical buildings.

16.2 Future Scope & Extensions

- **Real GPS / Beacon Integration:** Live blue-dot indoor positioning via Bluetooth Low Energy (BLE) beacons.
- **Accessibility-First Routing Mode:** Automatic path filtering ensuring wheelchair ramp and elevator-only navigation routes.

- **Multi-Stop Travelling Salesperson (TSP) Routing:** Finding the shortest tour through multiple departmental waypoints in a single student trip.

16.3 Academic Conclusion

The **SmartCampus** project demonstrates a complete, rigorous realization of graph theory and greedy algorithmic optimization. By coupling Dijkstra's Algorithm with an efficient Binary Min-Heap Priority Queue, the system achieves provably optimal physical routing with $\mathcal{O}((V + E) \log V)$ performance. The integrated step visualizer and comparative benchmark matrix make the underlying data structure mutations clear and accessible for academic DAA evaluation.

Prepared by: DAA Student Group
Department: Computer Science & Engineering

Evaluated by: Course Faculty Coordinator
CHRIST (Deemed to be University)